Highlights

A Memento-, Hope-, and Reflection-Enhanced Large Language Model Pipeline for Simulated Joint-Operation Plan Generation and Evaluation

Changrui Zhang, Chengwei Yang

- Repository-grounded elsarticle draft for simulated joint-operation planning
- Full pipeline improves mission success over the pure LLM baseline
- Five-scenario scene-out evaluation shows Full beating Pure in 5/5 cases
- Draft reports only code-supported mechanisms and source-traceable results

A Memento-, Hope-, and Reflection-Enhanced Large Language Model Pipeline for Simulated Joint-Operation Plan Generation and Evaluation

Changrui Zhang[a], Chengwei Yang[a,*]

[a] *Beijing Institute of Technology, Beijing, China*

Abstract

This paper presents an evidence-grounded English manuscript draft for a simulation-oriented planning pipeline built around three implemented components: Memento-style case retrieval, a Hope adaptive weighting controller, and a reflection loop for iterative prompt refinement. The system generates structured joint-operation plans, evaluates them through a five-stage kill-chain simulation model, and exports the selected plan into DoDAF OV-5b, DoDAF OV-6c, and C2SIM artifacts. To avoid unsupported claims, the draft only describes mechanisms that are explicitly present in the current codebase and experiment outputs.

The main single-scenario ablation study uses the repository's sample coastal joint-assault scenario with 10 optimization iterations, 50 Monte Carlo simulation runs, a fixed random seed of 7, and writeback disabled. Under this setting, the full pipeline improves mission success from 0.6207 to 0.6740, overall effectiveness from 0.7186 to 0.8086, and the loss-exchange ratio from 1.5233 to 2.4617 relative to the pure large language model baseline. The main cross-scenario generalization study uses five scenario families and leave-one-source-scenario-out case banks derived from 250 source cases. In that evidence set, the full pipeline outperforms the pure baseline in all five scenarios, increasing average mission success from 0.6506 to 0.6793 and average overall effectiveness from 0.7302 to 0.7936.

The draft also documents current limitations, including the absence of machine-readable backend and hardware metadata in experiment outputs,

*Corresponding author.

Email address: yangchengwei@bit.edu.cn (Chengwei Yang)

the lack of a validated graphical abstract, and the absence of a fully verified LaTeX build on the current machine.

Keywords: large language models, case-based reasoning, simulation-based evaluation, adaptive weighting, structured planning

---

1. Introduction

Large language models (LLMs) can generate multi-step plans in a flexible and interactive manner, which makes them attractive for simulation-centric decision support and scenario exploration workflows [1]. At the same time, planning quality remains sensitive to context selection, prior experience, and iterative self-correction. Repository-local evidence for the current project suggests that three design directions are central to its implementation: retrieval-style memory augmentation, adaptive capability weighting, and reflection-driven prompt adjustment.

The memory side of the project is closer to case-based reasoning (CBR) and retrieval-augmented generation than to standalone prompting. The codebase stores structured cases, retrieves them through a FAISS-backed semantic index when available, and falls back to keyword retrieval otherwise [2, 3, 4, 5, 6]. The reflection side is closer to iterative self-improvement schemes in LLM agents, although the current implementation should be interpreted strictly as a repository-specific planning loop rather than as a reimplementation of any single external framework [7, 8].

Within this repository, the target problem is not free-form text generation. Instead, the system aims to produce structured joint-operation plans that can be simulated, compared under ablation settings, and exported to architecture and interoperability artifacts. The simulation layer scores plans through a five-stage kill-chain abstraction, computes mission-level metrics such as mission success and overall effectiveness, and aggregates repeated stochastic runs into Monte Carlo summaries. The export layer produces DoDAF and C2SIM outputs aligned with repository-local structured plan objects [9, 10].

This draft intentionally stays close to what can be verified from files in the current project. It does not claim formal guarantees, external benchmark coverage, or a complete literature survey. Instead, the paper makes four narrow contributions grounded in code and experiment outputs:

1. It documents the repository's planning pipeline as an integrated workflow that combines case retrieval, adaptive weighting, reflection, structured simulation evaluation, and standards-oriented exporters.
2. It reports a single-scenario four-way ablation using the repository's canonical ablation suite.
3. It reports a five-scenario cross-scenario generalization study using the repository's canonical scene-out evaluation suite.
4. It surfaces practical limitations that remain before journal submission, including the lack of a machine-readable runtime manifest, unfinished graphical-submission assets, and remaining editorial metadata.

The rest of the paper describes the implemented pipeline, the selected evidence scope, the main quantitative findings, and the remaining submission gaps.

2. Method

The repository is organized around structured scenario, plan, simulation, and export objects rather than free-form prompting alone. This section deliberately limits itself to mechanisms that are directly supported by the current code and stored outputs.

2.1. Pipeline overview

Figure 1 presents the overall framework of the implemented planning pipeline. The framework starts from a structured scenario representation and couples two auxiliary mechanisms before each generation step: a Memento memory module that retrieves and filters prior cases, and a Hope adaptive controller that transforms scenario attributes into a bounded capability state. These two information sources, together with optional reflection patches from the previous iteration, are fused during prompt construction for LLM-based structured plan generation. The resulting candidate plan is then evaluated by the simulation layer through the repository's five-stage kill-chain abstraction, which yields mission-level metrics and stage-level deficits. These evaluation signals are used in two feedback paths: one path updates the adaptive capability state, and the other path produces reflection guidance for the next generation round. After the iterative loop terminates, the highest-scoring plan is selected and exported into the downstream structured artifacts used by the repository.

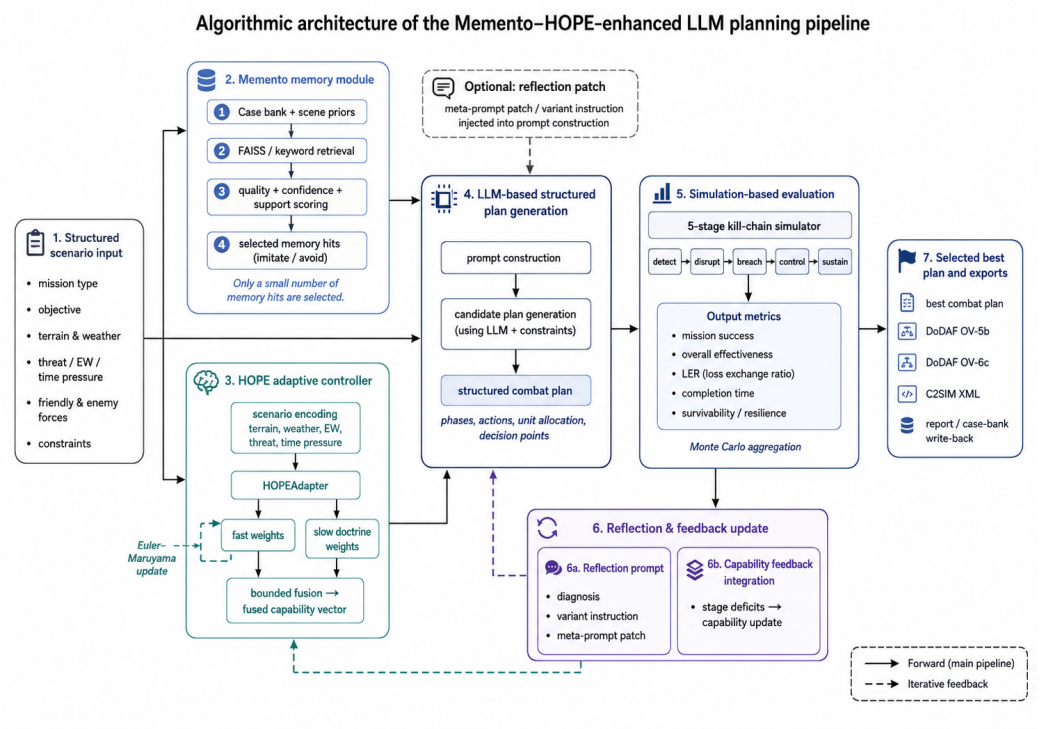

Figure 1: Overall framework of the Memento–HOPE-enhanced LLM planning pipeline. The framework maps structured scenario input to four tightly connected stages: memory retrieval and selection, adaptive capability modulation, LLM-based structured plan generation, and simulation-based evaluation. The solid arrows denote the main forward computation from scenario encoding to best-plan selection and export, whereas the dashed arrows denote the iterative feedback paths that inject reflection guidance and capability updates into subsequent planning rounds.

2.2. 1. Structured scenario input and plan representation

The domain layer defines normalized capability dimensions (`fires`, `mobility`, `protection`, `awareness`, `ew`, `sustainment`, and `c2`), typed actions, force units, scenarios, plan phases, and complete combat plans. These objects enforce non-empty text fields, bounded numeric ratios, and structured phase/action serialization. The downstream simulator and exporters therefore operate on typed fields rather than on unconstrained narrative text.

2.3. 2. Memento memory module: case retrieval and selection

The case-memory module stores records with mission type, terrain, objective, roles, key actions, lessons, outcomes, and metric summaries. When optional embedding dependencies are available, the module uses a Sentence-Transformer encoder with a FAISS index; otherwise it falls back to keyword retrieval [4, 5, 6, 2, 3]. Retrieved items are scored by confidence, blended with a case-quality heuristic, and labeled as either *imitate* or *avoid* according to the source outcome.

The pipeline does not pass all retrieved cases directly to the generator. Instead, it computes a scenario-dependent support score, sorts candidates by support, quality, and confidence, and keeps at most two selected memory hits in strict mode. For sparse generalization settings, the pipeline can also inject hand-authored scene priors for coastal assault, mountain reconnaissance, maritime sustainment, and urban defense scenes. These priors are represented as standard case records and enter the same scoring and selection path.

2.4. 3. HOPE adaptive controller: scenario encoding, fast weights, and bounded fusion

The adaptive controller maintains a slow capability-weight vector, a fast adjustment vector, and a neural adapter named `HOPEAdapter`. The scenario encoder converts terrain and weather into one-hot vectors and appends three scalar features: electronic-warfare threat, threat level, and time pressure. The adapter contains an input gate, a candidate projection, an output projection, and a value estimator that modulates a learned forget gate. In implementation terms, the forget gate depends on an estimated value term and a log-age term, which allows the hidden state to be refreshed during feedback updates while remaining stable during inference.

Fast weights are updated by an Euler–Maruyama-style step:

$$W_{\text{fast}} \leftarrow W_{\text{fast}} + \theta(\hat{W}_{\text{fast}} - W_{\text{fast}})\Delta t + \epsilon\sqrt{\Delta t}\,\xi,$$

where the target vector $\hat{W}_{\text{fast}}$ comes from the adapter, θ is a recovery coefficient, and ϵ controls the noise term. The resulting fast weights are clipped to scenario-dependent lower and upper bounds. The fused capability vector is then formed by adding a scaled fast component to the slow weights and enforcing mission- and threat-specific floors on key dimensions such as fires, mobility, awareness, and C2.

2.5. 4. LLM-based structured plan generation

The plan-generation stage receives three classes of conditioning signals from the upstream modules: the structured scenario description, the selected memory hits labeled as *imitate* or *avoid*, and the fused capability state computed by the Hope controller. These elements are injected into prompt construction together with repository-local planning constraints and any optional reflection patch. The LLM is then asked to generate a candidate combat plan in a structured format rather than unconstrained free text.

The resulting plan is normalized into typed phases, actions, unit allocations, and decision points so that downstream evaluation and export can operate on stable fields. In this sense, the LLM stage is not treated as a standalone text generator; it acts as the proposal engine inside a larger structured loop whose inputs and outputs are both repository-defined objects.

2.6. 5. Simulation-based evaluation

The simulator evaluates each plan phase against a five-stage kill-chain model. Stage scores are combined into phase success, mission success, loss-exchange ratio (LER), survivability, command resilience, completion time, and an overall effectiveness score. Under repeated stochastic evaluation, the pipeline aggregates multiple simulation runs and stores the mean, standard deviation, minimum, maximum, and 95% confidence interval for selected metrics [11].

2.7. 6. Reflection and capability feedback update

After each planning iteration, the controller derives stage deficits from the simulator's five-stage scores (`detect`, `disrupt`, `breach`, `control`, and `sustain`). These deficits are projected onto capability dimensions through fixed stage-to-capability mappings and an additional bottleneck-boost term. The adapter is then trained with a smooth-L_1 objective toward a bounded desired fast-weight target. Slow weights are updated through a bounded transformation that depends on the utility gradient, a delay-correction term, and minimum floors on selected offensive dimensions. The current code therefore supports adaptive, bounded, feedback-driven weight updates, but this draft does not infer a formally verified optimizer or theorem-backed convergence guarantee from those implementation choices.

Reflection is handled separately from the adaptive weight update. When enabled, the pipeline calls a reflection prompt that summarizes diagnosis items, variant instructions, and a meta-prompt patch for the next planning

Table 1: Representative cross-standard export correspondence for the Full-setting coastal joint-assault example under the selected scene-out suite. The table is derived from `best_plan.json`, `dodaf_ov5b.json`, `dodaf_ov6c.json`, and `c2sim.xml` in the same result directory.

Representative element	Structured plan object	OV-5b	OV-6c	C2SIM
Operation header	Top-level plan title, mission type, commander intent, desired end state, and control rules	`operation_name`	`operation_name`	`Header/Order` fields
Phase 1	`phases[0]` (`phase_id = P1`); actions: `recon`, `c2`	`OV5B-ACT-01`	`OV6C-EVT-01`	`Task P1`
Phase 2	`phases[1]` (`phase_id = P2`); actions: `strike`, `mobility`, `ew`	`OV5B-ACT-02`	`OV6C-EVT-02`	`Task P2`
Phase 3	`phases[2]` (`phase_id = P3`); actions: `recon`, `strike`, `mobility`	`OV5B-ACT-03`	`OV6C-EVT-03`	`Task P3`
Phase 4	`phases[3]` (`phase_id = P4`); actions: `control`, `sustain`	`OV5B-ACT-04`	`OV6C-EVT-04`	`Task P4`
Post-plan feedback	Simulation assessment metrics and writeback trigger	–	`OV6C-EVT-05`	–

round. Reflection can be suppressed for some scene types depending on scenario properties and memory support [7, 8].

2.8. 7. Best-plan selection and standards-oriented export

The same selected plan can also be exported into repository-local DoDAF OV-5b, DoDAF OV-6c, and C2SIM representations [9, 10]. These exporters are deterministic transformations from structured plan fields to machine-readable JSON or XML artifacts, which helps connect the narrative plan output to downstream modeling and interoperability workflows.

Table 1 gives a representative export example from the selected Full-setting coastal joint-assault result bundle. The key point is that the repository does not export an unrelated post-processed summary: the same structured plan header and phase objects are preserved across the OV-5b activity view, the OV-6c event-trace view, and the C2SIM order representation, with a final OV-6c closure event used to attach simulation feedback to the next planning round.

3. Experimental Setup

This draft uses a single canonical evidence scope to avoid mixing results from incompatible output directories. The goal is to keep the first English manuscript internally consistent and fully traceable to repository-local files.

3.1. Evidence scope and repository constraints

The selected single-scenario ablation evidence comes from the canonical ablation suite (`result/ablation_suite_seed7_v3`). The selected cross-scenario generalization evidence comes from the canonical scene-out suite (`result/generalization_suite_seed7_v2_sceneout_iter20`). The manuscript excludes other result directories from the main body and treats them as audit-only artifacts.

The sample ablation scenario is a coastal-urban assault case with rain, threat level 0.74, electronic-warfare threat 0.68, time pressure 0.77, five friendly units, and three enemy units. Its objective is to seize a coastal landing window and suppress enemy air-defense and shore-defense nodes for follow-on amphibious expansion. The frozen seed case bank used by the ablation suite contains five records.

The generalization study uses five repository-local scenarios listed in `data/generalization_scenarios/manifest.json`: coastal assault, urban defense, mountain reconnaissance, river crossing assault, and maritime sustainment. The selected evidence set relies on `data/generalization_case_banks_v2_sceneout/summary.json`, which records 250 source cases and five leave-one-source-scenario-out case banks with 200 cases each.

3.2. Ablation configuration

The ablation suite is generated by `run_ablation_suite.ps1` and summarized by `military_research/summarize_ablations.py`. In the canonical evidence set, all four ablation groups use the same random seed (7), the same sample scenario, 10 planning iterations, and 50 Monte Carlo simulation runs, with writeback disabled in the recorded experiment configuration. The four compared settings are:

- **Pure LLM**: memory, Hope, and reflection are disabled;
- **LLM + Memento**: memory is enabled, Hope and reflection are disabled;
- **LLM + Hope**: Hope is enabled, memory and reflection are disabled;
- **Full**: memory, Hope, and reflection are all enabled.

3.3. Cross-scenario generalization configuration

The generalization suite is generated by `run_generalization_suite.ps1`, which calls the ablation runner for each scenario and then summarizes the suite with `military_research/summarize_generalization.py`. In the selected evidence set, each scenario uses 20 planning iterations, 50 Monte Carlo simulation runs, random seed 7, writeback disabled, and a scene-specific leave-one-source-scenario-out case bank.

Table 2 summarizes the five repository-local scenarios used in the selected scene-out suite. The table keeps only the scenario attributes that matter most

Table 2: Overview of the five scenario families used in the selected scene-out generalization suite. Threat, EW, and time are the normalized scenario coefficients stored in the repository-local JSON files. Each scene-out case bank contains 200 cases derived from the same 250-case source pool with the target scenario held out.

Scenario	Family	Terrain / weather	Operational focus	Friendly / enemy units	Threat	EW	Time
Coastal joint assault	assault	coastal-urban / rain	Seize a landing window and suppress shore and air defense for follow-on expansion.	5 / 3	0.74	0.68	0.77
Urban hub defense	defense	urban / fog	Hold the transport and communications hub against an armored urban thrust.	4 / 2	0.69	0.54	0.66
Mountain corridor reconnaissance	recon	mountain / cloudy	Collect high-value fire-node and supply-line intelligence under constrained visibility.	3 / 2	0.58	0.46	0.55
River crossing breakthrough	assault	river-plain / overcast	Establish a defended crossing window and secure the bridgehead for follow-on forces.	4 / 2	0.71	0.57	0.73
Island resupply corridor	sustain	maritime-island / windy	Keep the island-chain resupply route open under persistent harassment and interdiction.	4 / 2	0.63	0.59	0.61

Table 3: Definitions of the main quantitative metrics reported in the manuscript. The wording follows the repository's simulator implementation rather than an external benchmark specification.

Metric	Repository-local meaning and code basis	Preferred direction	Stored field
Mission success	Mission-level success score. Computed from a weighted combination of stage-based success across the five kill-chain stages and average phase success, with extra penalties when C2 is weak or time pressure is extreme.	Higher	`mission_success`
Overall effectiveness	Composite utility score for overall plan quality. Combines mission success, normalized LER, survivability, command resilience, and a bounded completion-time term into one summary score.	Higher	`overall_effectiveness`
Loss-exchange ratio (LER)	Enemy-loss to friendly-loss ratio under the simulator's phase-by-phase loss accounting. Values above 1 indicate that the plan inflicts more loss on the opposing side than it absorbs.	Higher	`ler`
Command resilience	Proxy for the plan's ability to maintain command continuity under pressure. Computed from the fused C2, EW, and awareness weights, with a subtraction term for scenario EW threat.	Higher	`command_resilience`
Completion time	Total simulated execution time of the plan in hours. This value also enters the overall-effectiveness score through a bounded time term, so slower plans are penalized both as a standalone metric and inside the composite score.	Lower	`completion_time_hours`

for experimental reading: mission family, terrain and weather, operational focus, force size, and the three normalized pressure coefficients that feed the planning pipeline.

3.4. Metrics

The paper reports the same metrics stored in `full_result.json`: mission success, overall effectiveness, loss-exchange ratio (LER), completion time in hours, command resilience, stage scores for the five kill-chain stages, and Monte Carlo confidence intervals where available. Table 3 summarizes the five headline metrics used throughout the main result tables and figures.

In implementation terms, the current code computes mission success from a weighted combination of stage-level and phase-level success, while overall effectiveness combines mission success, LER, survivability, command resilience, and a time penalty. For the Monte Carlo summaries, the reported headline values in the manuscript are arithmetic means over the repeated simulation

runs, and confidence intervals are reported where the stored result object provides them.

3.5. Runtime metadata

The experiment owner has confirmed that the selected manuscript evidence was produced with the local backend model deepseek-r1-0528-qwen3-8b inside the Python virtual environment `Memento`. The reported hardware is a 13th Gen Intel(R) Core(TM) i7-13700F CPU with an NVIDIA GeForce RTX 3060 GPU. The selected `full_result.json` files do not themselves persist that backend identifier or hardware profile, so the draft treats them as author-supplied experimental metadata rather than as fields extracted from the raw result objects.

3.6. Reproducibility gaps

Two gaps remain important for a submission-ready version. First, the current manuscript can now name the confirmed backend model, virtual environment, CPU, and GPU, but the stored result files still do not record a machine-readable runtime manifest or operating-system details. Second, the current PDF can be compiled with the workspace-local TinyTeX toolchain, but the final submission package still needs the remaining editorial cleanup, including any final acknowledgments wording, data-availability wording, graphical-submission assets, and journal-specific packaging checks. These missing pieces are documented explicitly in the manuscript notes rather than being guessed.

4. Results

4.1. Single-scenario ablation

Table 4 reports the main ablation metrics for the canonical single-scenario evidence set. All values in this subsection are copied from the corresponding `full_result.json` files in the canonical ablation suite. Relative to the pure LLM baseline, the full pipeline improves mission success by 0.0533, overall effectiveness by 0.0900, and LER by 0.9384. The Hope-only variant already provides a visible gain over the baseline, whereas the memory-only variant produces only a modest positive shift in mission success and overall effectiveness.

The stage-level scores in Table 5 help explain where the improvement is concentrated. In the selected evidence set, the full system is strongest

Table 4: Main ablation results on the sample coastal joint-assault scenario. All groups use 10 planning iterations, 50 Monte Carlo runs, seed 7, and writeback disabled.

Setting	Mission success	Overall effectiveness	LER	Time (h)	Command resilience
Pure LLM	0.6207	0.7186	1.5233	11.42	0.6096
LLM + Memento	0.6225	0.7278	1.6164	11.36	0.6096
LLM + Hope	0.6414	0.7849	1.9360	9.91	0.7455
Full	0.6740	0.8086	2.4617	9.94	0.7812

Table 5: Five-stage kill-chain scores for the ablation study.

Stage	Pure LLM	LLM + Memento	LLM + Hope	Full
detect	0.5938	0.6232	0.6575	0.6912
disrupt	0.6022	0.5997	0.6399	0.6393
breach	0.5835	0.5759	0.5778	0.6159
control	0.6134	0.6271	0.6445	0.6996
sustain	0.6157	0.6611	0.6560	0.6926

on `control` and `sustain`, while `breach` remains its weakest stage. This is consistent with the controller's feedback path, which reacts to stage deficits and bottlenecks, although the stored results alone do not establish a causal decomposition of the gain.

Figure 2 presents the same five-stage evidence in a more comparison-oriented form. Two patterns are visually stable across all four settings. First, `breach` is the lowest-scoring stage throughout the ablation, which makes it the most persistent bottleneck in this sample scenario. Second, the full pipeline improves all five stages relative to the pure baseline, but the gain is uneven: the clearest recovery appears in `control` and `detect`, whereas the residual weakness of `breach` and the flatter movement of `disrupt` indicate where the current pipeline remains most fragile.

4.2. Cross-scenario generalization

The selected generalization evidence covers five scenario families. In this set, the full pipeline outperforms the pure baseline in all five scenarios and is not lower than the pure, Memento-only, and Hope-only variants in four of the five scenarios. Averaged across the suite, mission success rises from 0.6506 to 0.6793 and overall effectiveness rises from 0.7302 to 0.7936. These summary values are taken from the canonical scene-out summary and cross-checked against the scene-level `full_result.json` files.

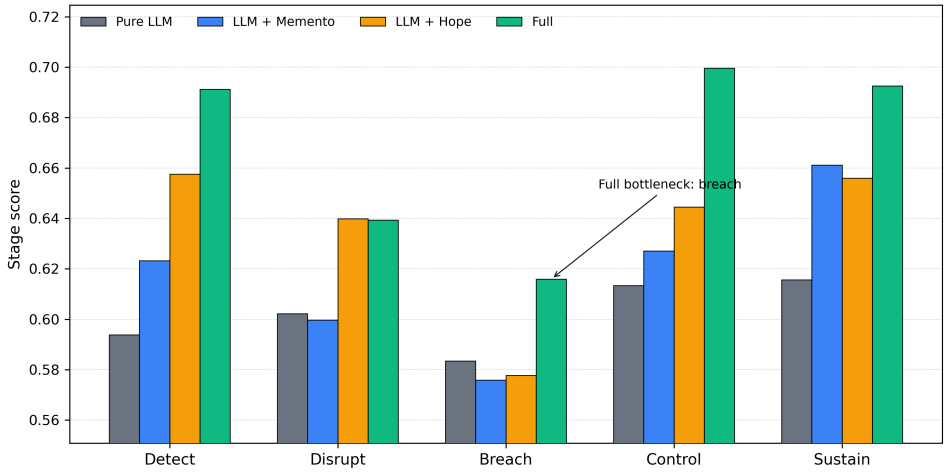

Figure 2: Five-stage kill-chain scores for the canonical single-scenario ablation. The figure visualizes the same values reported in Table 5. Across all four settings, `breach` remains the lowest stage, while the Full pipeline shows its largest relative gains in `detect`, `control`, and `sustain`. The annotation marks the weakest stage of the Full setting in order to make the residual bottleneck explicit.

Table 7 shows the scenario-level results for the full pipeline and the pure baseline, together with the mission-success confidence intervals reported by the stored Monte Carlo summaries. The largest mission-success gain appears in the urban defense scenario (+0.0453), while the smallest gain appears in the river-crossing scenario (+0.0098). Even when the mission-success gain is modest, the full system still improves overall effectiveness in every selected scenario in the chosen evidence set.

Figure 3 visualizes the same cross-scenario evidence from a scene-centered angle. Each panel corresponds to one scenario and compares the four ablation settings directly. The Full pipeline remains the highest bar in all five panels, whereas the relative ordering of the Memento-only and Hope-only variants changes with scenario family.

Figure 4 is an auxiliary convergence-style visualization derived from the 50-iteration Full-setting suite. For each iteration index, the figure averages the Full-pipeline metric values over the five scenario families and therefore should not be mixed numerically with the 20-iteration tables above. It is included to show the longer-run tendency of mission success, overall effectiveness, LER, completion time, and command resilience more clearly.

Table 6: Cross-scenario summary for the selected leave-one-source-scenario-out evidence set.

Summary item	Pure LLM	Full
Scenario count	5	5
Average mission success	0.6506	0.6793
Average overall effectiveness	0.7302	0.7936
Full wins against Pure LLM	–	5/5
Full not lower than Pure/Memento/Hope	–	4/5

Table 7: Scenario-level generalization results for the selected evidence set.

Scenario	Family	Terrain	Pure MS	Full MS	Full − Pure MS	Full MS 95% CI
Coastal joint assault	assault	coastal	0.6074	0.6359	+0.0285	[0.6352, 0.6366]
Island resupply corridor	sustain	maritime	0.6273	0.6764	+0.0491	[0.6757, 0.6771]
Mountain corridor reconnaissance	recon	mountain	0.6931	0.7039	+0.0108	[0.7031, 0.7047]
River crossing breakthrough	assault	river	0.6634	0.6732	+0.0098	[0.6724, 0.6740]
Urban hub defense	defense	urban	0.6619	0.7072	+0.0453	[0.7065, 0.7080]

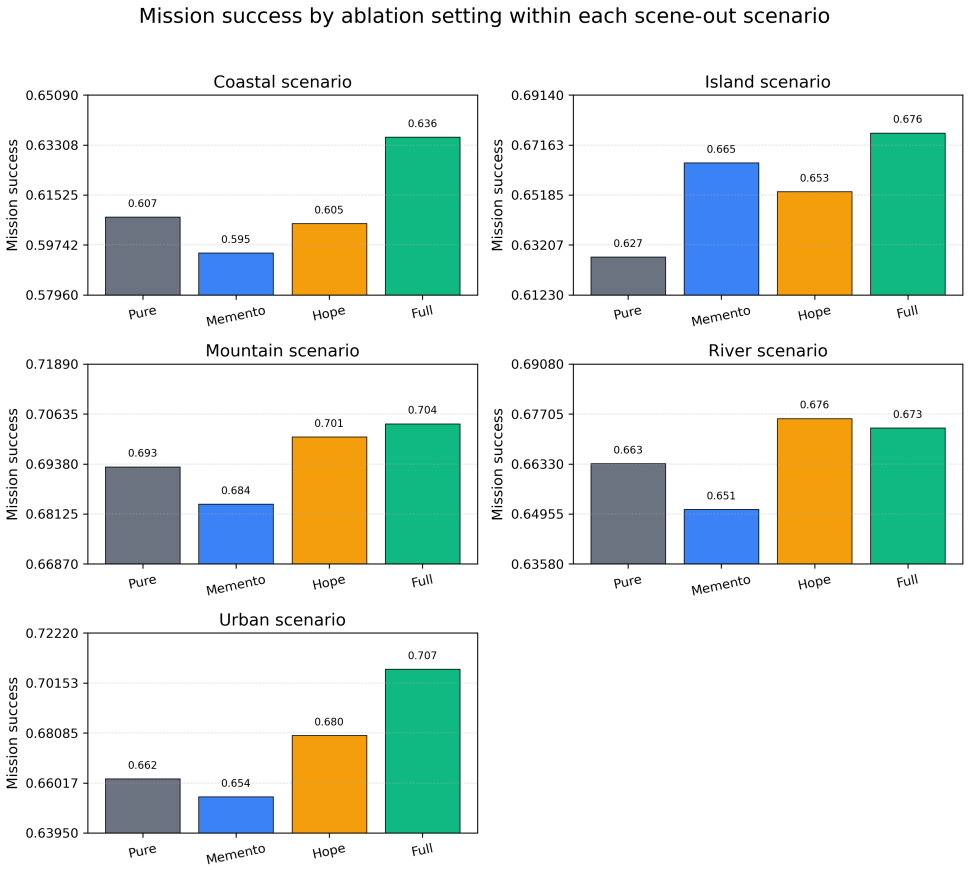

Figure 3: Mission success by ablation setting within each of the five scene-out scenarios in the canonical generalization suite. Each panel uses a locally tightened y-axis range to make between-group differences easier to inspect. The plotted values come from the corresponding `best_simulation.mission_success` fields in the per-scene `full_result.json` files.

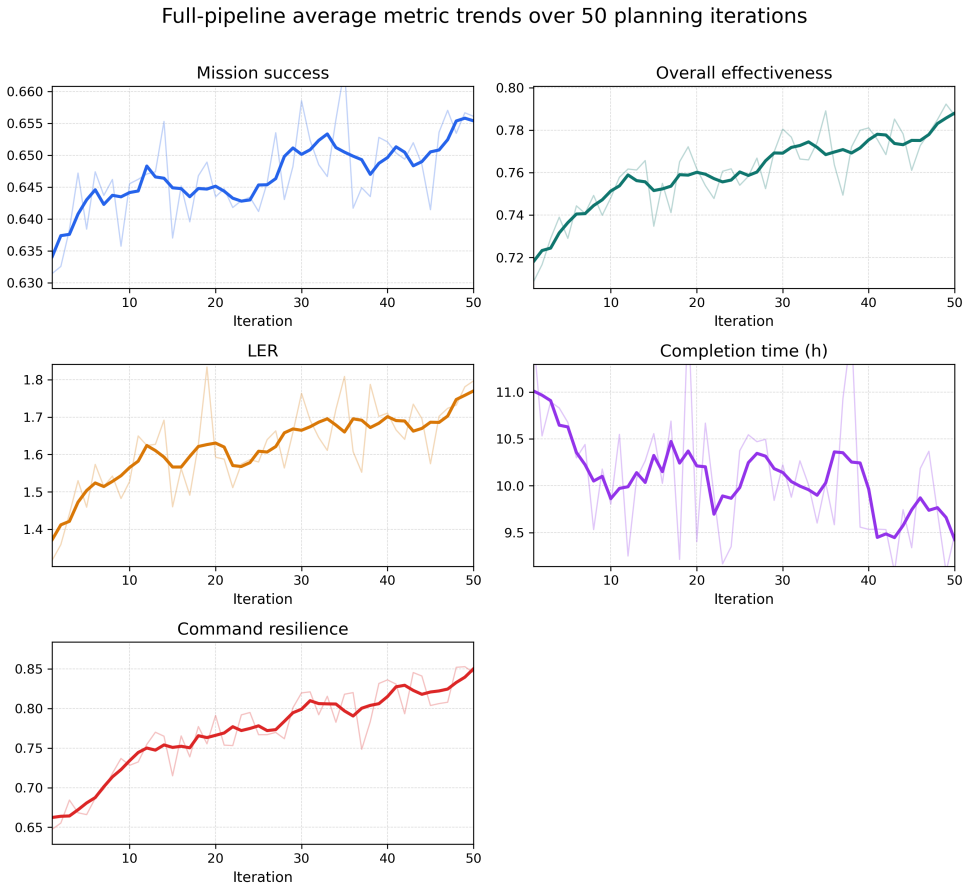

Figure 4: Average metric trajectories of the Full pipeline over 50 planning iterations. In each panel, the faint line shows the raw iteration-wise mean across the five Full-setting scenario runs, and the darker line shows a centered five-iteration moving average. Each panel also uses a tightened y-axis range to make the trend easier to inspect.

4.3. Interpretation

Two patterns are consistent across the selected evidence. First, adaptive weighting appears to be an important contributor: the Hope-only variant already improves several metrics substantially in the single-scenario ablation. The stage-level view in Table 5 and Figure 2 makes that contribution more specific. The full pipeline does not raise all kill-chain stages uniformly. Instead, it shows its clearest recovery in `detect`, `control`, and `sustain`, while `breach` remains the persistent bottleneck and `disrupt` improves only modestly. This pattern is consistent with the feedback-update logic described in the method section: the controller reacts to stage deficits and reallocates capability emphasis, but the stored result traces suggest that repeated reweighting is better at reinforcing already responsive stages than at fully removing the hardest breach-stage weakness in the current sample scenario.

Second, memory support becomes more valuable when the case-bank construction is broadened. The frozen seed bank used in the single-scenario ablation contains only five records, whereas the selected generalization evidence uses scene-out banks derived from 250 source cases. This difference plausibly contributes to the cleaner across-scenario gains observed in the selected generalization suite, but the current draft treats that interpretation as suggestive rather than proven.

At the same time, the results should be interpreted conservatively. The evidence supports improved simulation metrics inside the repository's evaluation framework; it does not, by itself, establish external validity, real-world operational value, or statistical significance beyond the stored Monte Carlo confidence summaries. No claim in this section should be read as an argument that the selected evidence set exhausts all result directories in the repository or that unreported suites would necessarily follow the same pattern.

5. Discussion

The current manuscript draft is intentionally conservative. It treats the repository as the primary source of truth and avoids overstating what the present evidence can support.

First, the strongest aspect of the evidence is the repository's internal consistency. The same structured plan objects feed the generator, simulator, result serializer, and DoDAF/C2SIM exporters. This makes it possible to trace reported numbers back to stored JSON files and to inspect the logic

that produced them. For a first journal draft, such traceability is more valuable than broader but weakly grounded claims.

Second, the selected evidence suggests that the interaction between memory augmentation, adaptive weighting, and reflection is useful within the current simulator. The full pipeline is best in the chosen single-scenario ablation and exceeds the pure baseline in all five scenarios of the chosen cross-scenario suite. The stage tables also show that the gains are not uniform: `breach` and `disrupt` remain recurring bottlenecks. This matters because it shifts the discussion away from headline averages alone and toward where the pipeline still appears fragile.

Several limitations remain before submission:

- **LLM backend traceability**: the experiment owner has confirmed the use of `deepseek-r1-0528-qwen3-8b` in the Python virtual environment `Memento` on a 13th Gen Intel(R) Core(TM) i7-13700F CPU with an NVIDIA GeForce RTX 3060 GPU, but the selected result files still do not store this information in a machine-readable runtime manifest.

- **Case-bank comparability**: the ablation evidence uses a five-record frozen seed bank, whereas the selected generalization evidence uses 250 source cases and 200-case scene-out banks. This is informative, but it also means that the role of memory quality and memory scale must be discussed carefully.

- **Figure scope**: the main-text figures are now generated directly from repository-local JSON files, but a graphical abstract and any additional publication-ready visual assets still remain to be prepared.

- **Metadata completeness**: the author list, affiliation, corresponding-author email, no-funding statement, competing-interest declaration, and CRediT roles are now recorded, but the final acknowledgments wording and data-availability wording may still need journal-specific refinement.

- **Build readiness**: the manuscript now compiles successfully with a workspace-local TinyTeX toolchain, but the final submission package still needs the remaining editorial metadata and any journal-specific packaging checks.

These limitations do not invalidate the draft; they define the next cleanup stage. The manuscript is already useful as a structured, source-traceable paper skeleton. The next iteration should focus on graphical-abstract preparation, final acknowledgments wording, data-availability wording, and the journal-specific submission package.

6. Conclusion

This paper draft documents a repository-implemented pipeline that combines case retrieval, adaptive capability weighting, reflection, structured simulation evaluation, and standards-oriented exporters for simulated joint-operation planning. Using a single canonical evidence scope, the draft reports that the Full Memento–Hope–Reflection configuration outperforms the pure LLM baseline in the selected ablation study and in all five scenarios of the selected cross-scenario generalization suite.

Just as importantly, the draft makes its current boundaries explicit. It does not claim formal proofs, external benchmark leadership, or evidence beyond what the repository can support. Instead, it establishes a clean Elsevier-style manuscript structure, a traceable evidence inventory, a figure set generated from repository-local JSON outputs, and a reproducible local PDF build. That gives the project a practical foundation for the next pass: polishing the final editorial metadata, refining the page layout, and preparing the journal-specific submission package.

CRediT authorship contribution statement

Changrui Zhang: Conceptualization, Methodology, Software, Formal analysis, Investigation, Data curation, Visualization, Writing - original draft. Chengwei Yang: Supervision, Validation, Resources, Writing - review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This research received no external funding. TODO: add any non-funding acknowledgments if applicable.

Data availability

The current draft is grounded in repository-local source files, experiment outputs, and manuscript notes under the project workspace. TODO: confirm whether a public data-and-code release statement can be provided for the selected case banks, result files, and manuscript assets.

References

[1] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, D. Amodei, Language models are few-shot learners, in: Advances in Neural Information Processing Systems, Vol. 33, 2020, pp. 1877–1901.

[2] A. Aamodt, E. Plaza, Case-based reasoning: Foundational issues, methodological variations, and system approaches, AI Communications 7 (1) (1994) 39–59.

[3] J. Kolodner, Case-Based Reasoning, Morgan Kaufmann, San Mateo, 1993.

[4] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Kuttler, M. Lewis, W.-t. Yih, T. Rocktaschel, S. Riedel, D. Kiela, Retrieval-augmented generation for knowledge-intensive nlp tasks, in: Advances in Neural Information Processing Systems, Vol. 33, 2020, pp. 9459–9474.

[5] N. Reimers, I. Gurevych, Sentence-bert: Sentence embeddings using siamese bert-networks, in: Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing, 2019, pp. 3982–3992.

[6] J. Johnson, M. Douze, H. Jegou, Billion-scale similarity search with gpus, IEEE Transactions on Big Data 7 (3) (2019) 535–547.

[7] N. Shinn, F. Cassano, E. Germanos, A. Gundersen, K. Narasimhan, S. Yao, Reflexion: Language agents with verbal reinforcement learning (2023). arXiv:2303.11366.
URL https://arxiv.org/abs/2303.11366

[8] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegreffe, U. Alon, N. Dziri, S. Prabhumoye, Y. Yang, M. Gupta, P. Clark, Y. Choi, H. Hajishirzi, A. Farhadi, A. Sabharwal, Self-refine: Iterative refinement with self-feedback (2023). arXiv:2303.17651.
URL https://arxiv.org/abs/2303.17651

[9] Department of Defense, DoD Architecture Framework Version 2.02, Washington, D.C.: Department of Defense (2010).

[10] Simulation Interoperability Standards Organization, SISO-STD-019-2020: Standard for Command and Control Systems-Simulation Systems Interoperation, Orlando: SISO (2020).

[11] A. M. Law, Simulation Modeling and Analysis, 5th Edition, McGraw-Hill Education, New York, 2015.